UNIVERSIDADE KIMPA VITA

INSTITUTO SUPERIOR POLITÉCNICO

RELATÓRIO TÉCNICO-CIENTÍFICO

Tema: *Sistema de Autenticação com Decoradores e Hash de Senhas*

Curso: Engenharia Informática

Ano de frequência: 3.º Ano

Período: Laboral

Grupo: 20

O ORIENTADOR

_____

MOYO KANIVENGIDIO

Abril, 2026

UNIVERSIDADE KIMPA VITA

INSTITUTO SUPERIOR POLITÉCNICO

RELATÓRIO TÉCNICO-CIENTÍFICO

Tema: *Sistema de Autenticação com Decoradores e Hash de Senhas*

Por:

António Dias Guilherme

António Mafuani Dikukumuka

Clemente José Canduanga

Jeremias Zeferino Maia

Luís Fernando José

Abril, 2026

1. Introdução

Este relatório apresenta a análise técnico-científica do sistema de autenticação desenvolvido em Python. O programa implementa mecanismos fundamentais de segurança informática, como hash de senhas, salt aleatório, controlo de acesso por perfis e uso de decoradores para proteger funções sensíveis. A explicação foi elaborada em blocos, com linguagem simples e progressiva, para que mesmo leitores sem conhecimento avançado em programação consigam compreender o funcionamento do sistema.

2. Objetivos

- **Principal**

Aplicar decoradores para controle de acesso e compreender a importância e implementação de hash de senhas para segurança em aplicações Python.

- **Especifico**

Permitir o cadastro, login, edição de perfil e administração de utilizadores, garantindo que apenas utilizadores autenticados possam aceder a determinadas funcionalidades. Além disso, o sistema diferencia permissões entre perfis comuns e administradores.

3. Estrutura Geral do Código

O código está organizado em blocos funcionais: importação de bibliotecas, segurança de senhas, cadastro de utilizadores, decoradores de autenticação, funções protegidas, login/logout e menu principal. Essa divisão melhora a legibilidade, manutenção e reutilização do software.

4. Explicação Detalhada por Blocos

Bloco A – Bibliotecas Importadas

O sistema utiliza hashlib para gerar hashes SHA-256, os para criar salts aleatórios, functools para preservar metadados das funções decoradas, e datetime para registar a hora de acesso. Essas bibliotecas são essenciais para segurança, organização e auditoria do sistema.

```
import hashlib
import os
import functools
from datetime import datetime
```

Fonte: do Autor

Bloco B – Variáveis Globais

A variável usuario_actual guarda o utilizador autenticado no momento. O dicionário usuarios funciona como base de dados em memória, armazenando nome, hash, salt e função de cada utilizador.

```
usuario_actual = None
usuarios = {}
```

Fonte: do Autor

Bloco C – Segurança com Hash e Salt

A função gerar_salt() cria uma sequência aleatória única para cada utilizador. A função hash_senha() combina a senha com o salt e aplica SHA-256, produzindo uma assinatura irreversível. Isso significa que o sistema nunca guarda a senha original, apenas a sua versão cifrada. A função verificar_senha() compara o hash gerado no momento do login com o hash guardado

.

```python
def gerar_salt():  2 usages
    # gera um salt aleatorio pra cada usuario
    return os.urandom(16).hex()

def hash_senha(senha, salt):  3 usages
    # mistura a senha com o salt e faz o hash SHA-256
    return hashlib.sha256((senha + salt).encode()).hexdigest()
```

Fonte: do Autor

Bloco D – Cadastro de Utilizadores

A função cadastrar_usuario() verifica se o nome já existe, gera salt, calcula o hash da senha e armazena os dados no dicionário. Este bloco demonstra boas práticas de validação de entrada e prevenção de duplicidade.

```python
# cadastro de usuarios

def cadastrar_usuario(nome_usuario, senha, funcao="usuario"):  1 usage
    if nome_usuario in usuarios:
        print(f"Esse usuario '{nome_usuario}' ja existe mano.")
        return False

    salt = gerar_salt()
    h = hash_senha(senha, salt)
    usuarios[nome_usuario] = {"hash": h, "salt": salt, "funcao": funcao}
    print(f"Usuario '{nome_usuario}' cadastrado com sucesso (funcao: {funcao}).")
    return True
```

Fonte: do Autor

Bloco E – Decoradores e Controlo de Acesso

O decorador @autenticado é um dos conceitos centrais do trabalho. Decoradores em Python são funções que modificam o comportamento de outras funções sem alterar o seu código interno. Neste caso, ele verifica se existe um utilizador logado e, opcionalmente, se o perfil possui a função necessária (por exemplo, admin). Assim, o controlo de acesso fica centralizado e reutilizável.

```python
#decorador de autenticacao

def autenticado(funcoes=None):  4 usages

    def decorador(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            global usuario_actual

            if usuario_actual is None:
                print("Acesso negado. Precisa fazer login primeiro.")
                return

            if funcoes is not None:
                if usuario_actual["funcao"] not in funcoes:
                    print(f"Sem permissao. Essa opcao e so pra: {funcoes}")
                    return

            return func(*args, **kwargs)
        return wrapper
    return decorador
```

Fonte: do Autor

Bloco F – Funções Protegidas

As funções ver_dados_sensiveis(), editar_perfil(), apagar_usuario() e listar_usuarios() estão protegidas pelo decorador. Isso garante que apenas utilizadores autorizados executem ações críticas. O administrador possui privilégios superiores, como apagar contas e listar todos os utilizadores. Exemplo com a função **apagar_usuario.**

```python
@autenticado(funcoes=["admin"])  1 usage
def apagar_usuario():
    print("\n--- Apagar Usuario (so admin) ---")
    alvo = input("Nome do usuario a apagar: ").strip()

    if alvo == usuario_actual["nome_usuario"]:
        print("Nao podes apagar a tua propria conta.")
        return

    if alvo in usuarios:
        del usuarios[alvo]
        print(f"Usuario '{alvo}' apagado.")
    else:...
```

Fonte: do Autor

Bloco G – Login e Logout

A função login() valida o nome do utilizador, verifica a senha usando hash e salt, e define o utilizador ativo. Já logout() encerra a sessão atual, removendo o acesso às áreas protegidas.

```python
def login(nome_usuario, senha):  1 usage
    global usuario_actual

    if nome_usuario not in usuarios:
        print("Nao achei esse usuario.")
        return False

    dados = usuarios[nome_usuario]
    if not verificar_senha(senha, dados["salt"], dados["hash"]):
        print("Senha errada.")
        return False

    usuario_actual = {"nome_usuario": nome_usuario, "funcao": dados["funcao"]}
    print(f"Login feito! Bem-vindo, {nome_usuario} ({dados['funcao']}).")
    return True
```

Fonte: do Autor

```python
def logout():  1 usage
    global usuario_actual
    if usuario_actual:
        print(f"Ate logo, {usuario_actual['nome_usuario']}.")
        usuario_actual = None
    else:
        print("Nao ta logado nenhum usuario.")
```

Fonte: do Autor

Bloco H – Menu Principal

O menu () implementa a interface textual do sistema. Utiliza um ciclo while True para manter o programa em execução até que o utilizador escolha sair. Cada opção chama uma funcionalidade específica, representando um fluxo simples e intuitivo.

```python
# ------ menu principal
windows
def menu():  1 usage


    while True:
        print("\n \n")
        if usuario_actual:
            print(f"  Logado: {usuario_actual['nome_usuario']} ({usuario_actual['funcao']})")
        else:
            print("  Nenhum usuario logado")
        print("\n \n")
        print("1 - Login")
        print("2 - Logout")
        print("3 - Ver dados sensiveis")
        print("4 - Editar perfil")
        print("5 - Apagar usuario  (so admin)")
        print("6 - Listar usuarios (so admin)")
        print("7 - Cadastrar novo usuario")
        print("0 - Sair")
```

5. Conceitos de Programação Utilizados

- ✓ Funções: blocos reutilizáveis de Código;
- ✓ Dicionários: estrutura de dados chave-valor para armazenar utilizadores;
- ✓ Decoradores: proteção e extensão de funções;

- ✓ Hashing: transformação irreversível de senhas;
- ✓ Salt: valor aleatório que reforça a segurança do hash;
- ✓ Estruturas condicionais: controlo lógico com if/elif/else;
- ✓ Ciclos de repetição: menu contínuo com while;
- ✓ Modularização: separação do sistema em blocos especializados.

6. Análise Científica e Boas Práticas

Do ponto de vista científico, o sistema demonstra princípios fundamentais de segurança da informação: confidencialidade, autenticação, autorização e integridade. O uso de SHA-256 com salt reduz o risco de ataques por tabelas rainbow. A aplicação de decoradores mostra domínio de abstração e reutilização de código, reduzindo redundância e aumentando a escalabilidade do software.

CONSIDERAÇÕES FINAIS

Conclui-se que o trabalho implementa corretamente um sistema de autenticação básico, mas robusto para fins académicos. Os conceitos aplicados são altamente relevantes em sistemas reais, como portais bancários, redes corporativas e plataformas web. A estrutura em blocos facilita o entendimento por iniciantes e demonstra maturidade técnica na organização do código.